Data Structure

2026-1학기 부경대학교 알고리즘 스터디

목차

- 정의
  - 자료구조
  - 추상적 자료형
- 선형 자료구조
  - Array
  - Linked List
  - Stack
  - Queue
- 비선형 자료구조
- 연관 배열
  - Set
  - Map
  - Hash Table

정의 - 자료구조

- 자료구조(data structure)란?
  - 데이터를 디스크(메모리)에 저장하고 조작하는 구조
  - 적절한 자료구조를 사용해 삽입, 삭제, 갱신, 탐색 등의 연산을 효율적으로 수행
- 자료구조를 배우는 이유?
  - 자료구조마다 제공하는 연산이 다르고, 같은 연산이더라도 시간 복잡도가 다름
  - 문제가 어떤 연산(삽입, 삭제, 대입, 질의 등)을 요구하는지에 따라 효과적인 자료구조가 존재하고, 적절한 자료구조를 사용해 시간 복잡도를 줄일 수 있음
  - **즉 자료구조를 공부한다는 것은 문제를 코드가 아니라 연산과 병목으로 바라보는 사고를 기르는 것!**

정의 - 추상적 자료형

- 추상적 자료형(ADT, Abstract Data Type)란?
  - 자료구조의 세부 구현은 은닉하고, 자료형이 제공하는 연산이 어떠한 기능을 하는지 수학적으로 정의한 자료형
  - 예) 스택의 경우 내부 구현에 상관없이 pop, push, top만 제공하면 되는 ADT
  - 가령 STL에서 제공하는 stack 컨테이너의 경우, 내부 구현의 디폴트는 deque
    - ※ std::stack<T, deque<T>> 가 기본 템플릿 인자이므로, vector<T> 혹은 list<T>로도 구현 가능
- 자료구조와의 관계?
  - ADT의 연산을 구현하기 위해 자료구조가 필요함
  - 자료구조는 추상적 자료형의 연산을 실제로 구현한 구현체가 됨

선형 자료구조

- 선형(線型)성 : 직선처럼 올곧은 성질을 가진 것
  - 수학적으로는 직선인 모양 (혹은 도형)을 의미함
- 선형 (linear) 자료구조란?
  - 데이터를 한 줄로 나열하여 저장하는 구조
  - 자료들의 앞, 뒤 관계가 **1 : 1** 구조
  - Array, Linked List 등이 선형 자료구조에 속함
  - 데이터간 단일한 순서성이 존재

선형 자료구조 - Array

- Array(배열)이란?
  - **같은 타입**의 데이터를 **연속된 메모리 공간**에 나란히 저장한 자료구조
  - 각 원소를 **인덱스**로 접근 가능
    - ※ C, C++에서는 첫번째 원소의 인덱스를 0으로 설정 (0-based Index)
  - 고정된 크기
    - ※ C++ STL에는 vector라는 동적 배열 타입이 존재
  - **논리적** 저장 순서와 **물리적** 저장 순서가 동일

Array

5	8	3	12	7
0	1	2	3	4

선형 자료구조 - Array

- Array 연산의 시간 복잡도
  - 길이가 N인 배열의 i번째 인덱스에 다음과 같은 연산을 수행한다고 가정해보자.
  - 접근 : O(1)
    - ※ 순차 접근할 필요 없이 인덱스에 해당하는 주소로 점프할 수 있기 때문
    - ※ 이러한 접근 방법을 임의 접근(random access) 라고 함
  - 갱신 : O(1)
  - 삽입 및 삭제 : O(N)
    - ※ 데이터를 앞, 중간 삽입할 경우 그 뒤의 원소들을 한칸씩 밀어야 하므로 O(N)의 시간 복잡도를 가짐
    - ※ 마찬가지로 데이터를 앞, 중간 삭제할 경우 그 뒤의 원소들을 한칸씩 당겨야 하므로 O(N)의 시간 복잡도를 가짐
    - ※ 크기가 고정적이므로 자료구조 차원에서 완전한 삽입 및 삭제 연산을 지원하지 않음

선형 자료구조 – Array

- 2차원 배열
  - 배열의 인덱스가 두 정수로 표현되는 배열
  - int arr[a][b]의 경우 길이가 b인 배열이 총 a개 모인 것(a * b)으로 볼 수 있음
  - 격자 모양의 배열로 a[i][j]와 같은 방법으로 접근 가능
  - 1차원 배열과 마찬가지로 메모리에 연속적으로 저장됨
    - ※ 언어 방식에 따라 행 우선인 경우 존재
  - 경우에 따라서 3차원 이상의 고차원 배열도 선언 가능

선형 자료구조 - Array

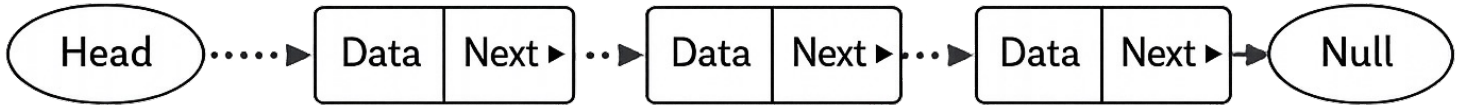
- **C++에서의 배열**
  - **배열이름[인덱스]** 로 원소에 접근 가능
  - 인덱스가 **0부터** 시작
  - **배열의 이름**이 대부분 배열의 **첫번째 원소의 포인터**를 반환
  - 가령 &arr[3]의 경우 (arr + 3)와 동일
  - 2차원 배열에서 &arr[3][2]의 경우 *(arr + 3) + 2와 동일

```cpp
int main() {
    // 크기가 5인 int형 배열 선언
    int A[5] = {1, 3, 5, 7, 9};


    // A[2]에 11 대입
    A[2] = 11;

    for(int i = 0 ; i < 5; i++)
        cout << A[i] << " ";

    cout << endl;

    // 크기가 3 x 3인 2차원 배열 선언
    int B[3][3] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };

    // 2차원 배열 접근후 값 대입
    B[2][1] = 0;

    for(int i = 0; i < 3; i++) {
        for(int j = 0; j < 3; j++) {
            cout << B[i][j] << " ";
        }
        cout << endl;
    }

}
```

```
1 3 11 7 9
1 2 3
4 5 6
7 0 9
```

선형 자료구조 – Linked List

- Linked List(연결 리스트)란?
  - **(값, 다음 노드의 주소)**를 하나의 노드로 저장해 순차를 유지하는 자료구조
  - 자료구조 차원에서 **삽입/삭제 연산** 제공
  - 논리적 선형성을 가지나, 데이터들의 **물리적인 위치는 연속적이지 않음**
  - 이전 원소의 주소를 포함하여 Double Linked List 자료구조도 표현 가능
  - Linked List의 시작 노드를 **Head**, 마지막 노드를 **Tail**이라고 함



선형 자료구조 – Linked List

- Linked List 연산의 시간 복잡도
  - 길이가 N인 Linked List의 i번째 노드에 다음과 같은 연산을 수행한다고 가정해보자
  - 접근 : O(N)
    - ※ i번째 노드의 주소 정보는 i – 1번째 노드에 저장되어 있음
    - ※ 따라서 head 노드에서부터 i – 1번째 노드까지 순서대로 접근해야 함
    - ※ 이러한 접근 방법을 순차 접근 (sequential access)라고 함
  - 갱신 : O(N)
    - ※ i번째 원소에 접근 (O(N)), 값 갱신 O(1)
  - 삽입 : O(N)
    - ※ i번째 원소에 접근 (O(N)), 인덱싱 O(1)
  - 삭제 : O(N)
    - ※ i번째 원소에 접근 (O(N)), 인덱싱 O(1)
  - 추가 : O(1)
    - ※ Tail 노드의 주소를 알 경우 O(1), 그렇지 않을 경우 삽입 연산과 동일

선형 자료구조 – Linked List

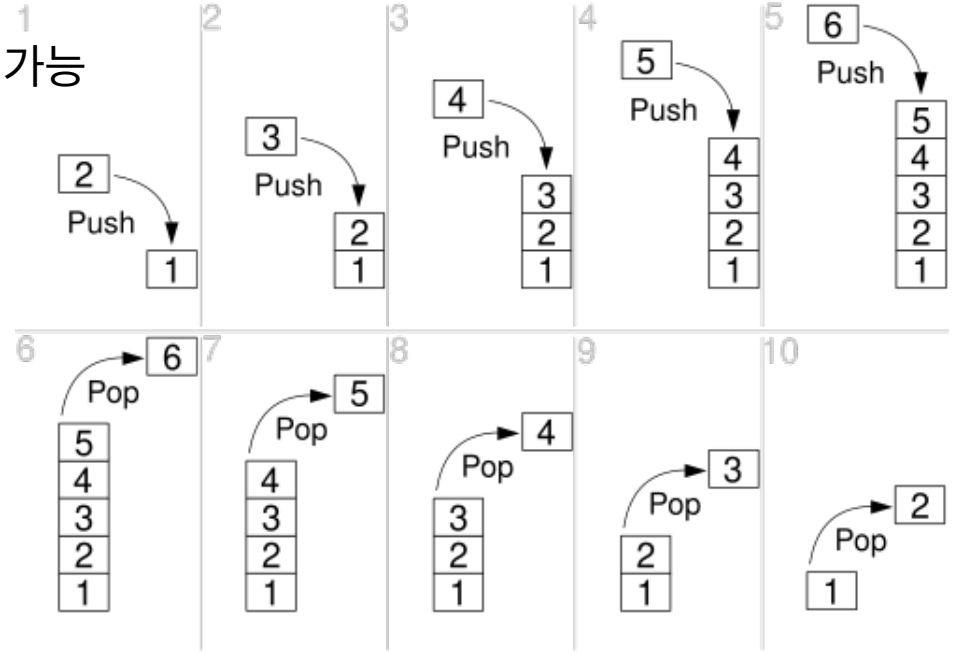
- C++ 에서의 Linked List
  - C++ STL의 std :: forward_list가 linked list로 구현되어 있음
  - 각 원소에 반복자 (iterator)로 접근
    - ※ 반복자에 대한 설명은 부록 Cpp for PS 참조
  - 범용적인 std::list를 주로 사용함
    - ※ List는 Linked List (혹은 Double Linked List) 또는 Array로 구현되는 추상 자료형
    - ※ std::list는 double linked list로 구현되어 있음

```
int main() {
    forward_list<int> linked_list;
    // 삽입
    // 1. 맨 앞 삽입
    linked_list.push_front(3);
    linked_list.push_front(5);
    linked_list.push_front(7);

    // 2. 중간 삽입
    // (반복자, 삽입할 값)을 인자로 받아 해당 반복자 다음에 삽입
    linked_list.insert_after(linked_list.begin(), 23);

    cout << "삽입 후 결과 : " << endl;
    for(auto i = linked_list.begin(); i != linked_list.end(); ++i)
        cout << *i << " ";
    cout << endl;

    // 삭제
    // 1. 맨 앞 삭제
    linked_list.pop_front();

    // 2. 반복자 값을 인자로 받아 해당 반복자 다음에 삭제
    linked_list.erase_after(linked_list.begin());

    cout << "삭제 후 결과 : " << endl;
    for(auto i = linked_list.begin(); i != linked_list.end(); ++i)
        cout << *i << " ";

    cout << endl;
}
```

```
삽입 후 결과 :
7 23 5 3
삭제 후 결과 :
23 3
```

선형 자료구조 - Stack

- Stack이란?
  - **LIFO** (Last-In, First-Out) 규칙을 가진 **추상적 자료형**
  - **Array** 혹은 **Linked List**와 같은 선형 자료구조로 쉽게 구현이 가능
- Stack 연산
  - push() : 스택에 값을 추가
  - pop() : 가장 최근에 추가된 값을 제거
  - peak() : 가장 최근에 추가된 값을 반환



선형 자료구조 - Stack

- PS에서의 Stack 응용
  - 간단한 구조 덕분에 다양하게 활용됨
  - 괄호 검증
    - ※ (), {} 와 같은 괄호식이 올바른지 스택으로 검증
  - 수식 표현식
    - ※ 연산자 우선순위를 스택을 통해 표현 후 처리할 수 있음
  - 깊이 우선 탐색
    - ※ 재귀 함수 대신 스택을 사용해 그래프에서 우선 순위 탐색 표현 가능
  - 그 외에도 프로세스의 콜 스택, 프로그래밍 언어의 구문 분석 등 다양한 분야에서 사용되는 ADT!

선형 자료구조 - Stack

- C++ 에서의 Stack
  - 템플릿 원형 : std::stack<T, Container>
    - ※ T는 스택에 사용할 데이터 타입, 컨테이너는 내부 구현체를 의미함
  - pop() : stack의 pop 연산을 제공
  - push() : stack의 push 연산을 제공
    - ※ pop(), push()의 시간 복잡도는 Container에 따라 상이하나, 주로 O(1)
  - top() : stack의 peak 연산을 제공
  - size() : 원소의 개수 리턴
  - empty() : stack의 원소의 개수가 0개면 true, 1개 이상이면 false를 리턴

```
int main() {
    stack<int> st;

    // 스택에 순서대로 1, 3, 5 추가
    st.push(1);
    st.push(3);
    st.push(5);

    // 출력
    while(!st.empty()) {
        cout << st.top() << " ";
        st.pop();
    }
}
```

```
5 3 1 %
```

제로 BOJ 10773

- 문제
  - K개의 정수가 순서대로 주어진다. 만약 정수가 0일 경우 가장 최근에 쓴 수를 지우고, 아닐 경우 해당 수를 쓴다. 이렇게 쓴 수들의 합을 구하라.

제로 BOJ 10773

- 문제
  - K개의 정수가 순서대로 주어진다. 만약 정수가 0일 경우 가장 최근에 쓴 수를 지우고, 아닐 경우 해당 수를 쓴다. 이렇게 쓴 수들의 합을 구하라.

- 해설
  - 입력이 0인 경우, 가장 최근에 추가된 수를 지운다. 즉 **마지막에 추가된 수를 지우는 LIFO 문제** -> 스택!
  - 0이 아닌 수가 들어오면 push, 0이 들어오면 pop
  - K개의 정수를 모두 확인한 다음, 스택에 남아있는 수를 더하여 출력하면 정답
  - 정수를 K번 push 혹은 pop 하며, 최대 K개의 수를 더해야 하므로 O(1 * K + a) = O(K)

```cpp
// BJ 10773 제로
#include <iostream>
#include <stack>

using namespace std;

int main() {
    cin.tie(0);
    ios::sync_with_stdio(false);

    // 수를 받아 적기 위한 stack 정의
    stack<int> st;

    // 수의 개수 K, 수의 총합 sum 정의 후 K 입력받기
    int K, sum = 0;
    cin >> K;

    // 총 K번 정수 입력받기
    while(K--) {
        int inpt;
        cin >> inpt;

        // 만약 입력이 0이면 최근에 적은 수 pop
        if(inpt == 0) {
            st.pop();
        }
        // 입력이 0이 아닌 정수라면 적어두기 (push)
        else {
            st.push(inpt);
        }
    }
    // 스택이 빌 때까지 적어놓은 수들을 합하기
    while(!st.empty()) {
        sum += st.top();
        st.pop();
    }

    cout << sum << endl;
}
```

괄호 BOJ 9072

- 문제
  - ‘(‘와 ‘)’로만 이루어진 문자열들이 주어졌을 때, 이 문자열들이 올바른 괄호 문자열인지 " YES” or “NO”로 출력하라

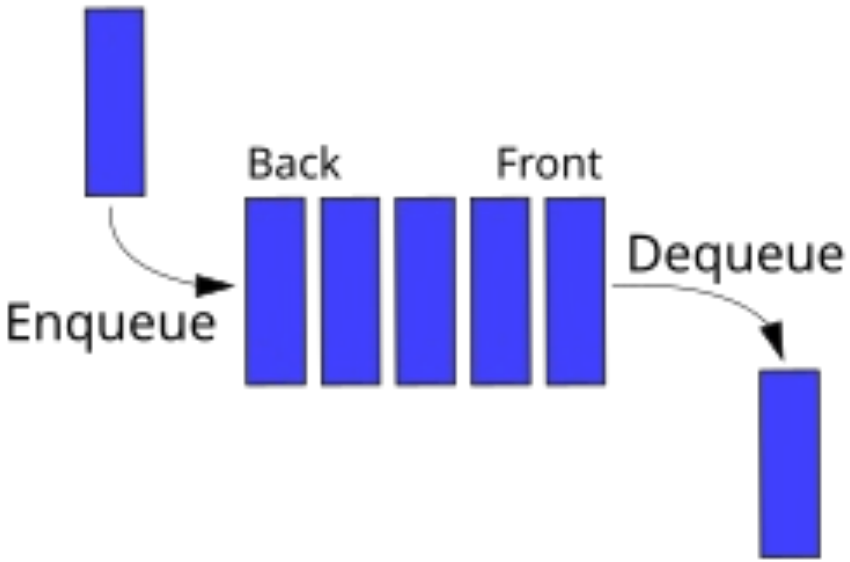
괄호 BOJ 9072

- 문제
  - ‘(‘와 ‘)’로만 이루어진 문자열들이 주어졌을 때, 이 문자열들이 올바른 괄호 문자열인지 " YES” or “NO”로 출력하라
- 풀이
  - 올바른 괄호문자열의 필요 조건은 무엇일까?
    - ※ 1) ‘(‘의 개수와 ‘)’의 개수가 동일하다
    - ※ 2) 왼쪽부터 읽었을 때 접두사에서, ‘)’의 개수가 ‘(‘개수보다 같거나 작다
  - 왼쪽부터 읽었을 때 ‘)’ 는 닫히지 않은 ‘(‘ 중 가장 **마지막**에 들어온 문자와 한 괄호를 구성
  - -> 스택에 ‘(‘문자열이 들어오면 push, ‘)’이 들어오면 pop함으로써 올바른 괄호 문자열인지 확인할 수 있음
  - 스택이 비었다면 현재까지의 문자열이 괄호를 이룬 상태이므로, 이때 ‘)’가 들어온다면 2)번 조건 불만족!
  - 문자열의 길이를 N이라고 했을 때, 총 N번 pop 혹은 push하므로 O(N)

```cpp
// BJ 9012 스택
#include <iostream>
#include <stack>
#include <string>

using namespace std;

int main() {
    cin.tie(0);
    ios::sync_with_stdio(false);

    // 테스트 데이터의 개수 T
    int T;
    cin >> T;

    while(T--) {
        // 스택에 문자 '('가 들어가므로 char형 stack 정의
        stack<char> st;

        // 문자열이 괄호문자열 (VPS)면 true, 아니면 false
        bool is_VPS = true;

        // 문자열 정의 후 입력받기
        string str;
        cin >> str;

        // 문자열의 첫번째 값 (왼쪽)부터 읽기
        for(int i = 0; i < str.size(); i++) {

            // 만약 문자가 '('면 스택에 push
            if(str[i] == '(') {
                st.push(str[i]);
            }
            else if(str[i] == ')') {

                // ')'일 때 스택이 비어있다면 ')'의 개수가  '('보다 많아지는 구간이 존재한다는 뜻이므로 NO 출력
                if(st.empty()) {
                    is_VPS = false;
                    break;
                }
                // 스택에 '('가 들어있다면 pop
                else {
                    st.pop();
                }
            }
        }

        // 중간에 break되거나, 마지막 문자열까지 확인했을 때 '('가 남아있는 경우라면 NO 출력
        if(!is_VPS || !st.empty())
            cout << "NO" << endl;
        else
            cout << "YES" << endl;
    }
}
```

선형 자료구조 - Queue

- Queue란?
  - **FIFO** (First-In, First-Out) 규칙을 가진 추상적 자료형
  - 데이터가 들어가는 위치를 **Back** 혹은 **Rear**로 표현
  - 데이터가 나가는 위치를 **Front**로 표현
- Queue 연산
  - enqueue() : 큐에 데이터를 삽입
  - dequeue() : 가장 마지막에 추가된 값 제거



선형 자료구조 - Queue

- C++ 에서의 Queue
  - 템플릿 원형 : std::queue<T, Container>
  - pop() : queue의 dequeue 연산을 제공
  - push() : queue의 enqueue 연산을 제공
    - ※ stack과 마찬가지로 pop, push의 연산이 컨테이너 별로 조금씩 상이하나, 주로 O(1) 수준으로 구현됨
  - back() : queue의 맨 뒤 원소 참조
  - front() : queue의 맨 앞 원소 참조
  - stack 과 동일하게 empty(), size() 함수 제공

```
int main() {
    queue<int> q;

    // 삽입
    q.push(1);
    q.push(2);
    q.push(3);

    // 큐가 빌 때까지 dequeue
    while(!q.empty()) {
        cout << q.front() <<  " ";
        q.pop();
    }
}
```

```
1 2 3 %
```

카드2 BOJ 2164

- 문제
  - 1, 2, ..., N번 카드가 N이 아래에 오도록 순서대로 놓여있다. 맨 위에서 한 장을 버리고, 그 다음 카드 한 장을 맨 아래에 놓는 과정을 카드가 한 장 남을 때까지 반복한다면, 마지막으로 남는 카드의 번호는?

카드2 BOJ 2164

- 문제
  - 1, 2, ..., N번 카드가 N이 아래에 오도록 순서대로 놓여있다. 맨 위에서 한 장을 버리고, 그 다음 카드 한 장을 맨 아래에 놓는 과정을 카드가 한 장 남을 때까지 반복한다면, 마지막으로 남는 카드의 번호는?
- 풀이
  - 카드가 빠지는 방향은 맨 위, 카드가 들어가는 방향은 맨 아래
  - 즉 카드의 맨 위를 front, 카드의 맨 아래를 back으로 두었을 때, front에서만 카드가 빠지고 back에서만 카드가 들어가므로 queue로 구현이 가능
  - 카드를 버리고, 다시 한 장을 꺼낸 다음, 맨 아래에 놓는 과정 (pop 2번, push 1번)에서 카드가 한 장씩 버려지므로, 최대 N 번 반복하게 됨 -> O(N)

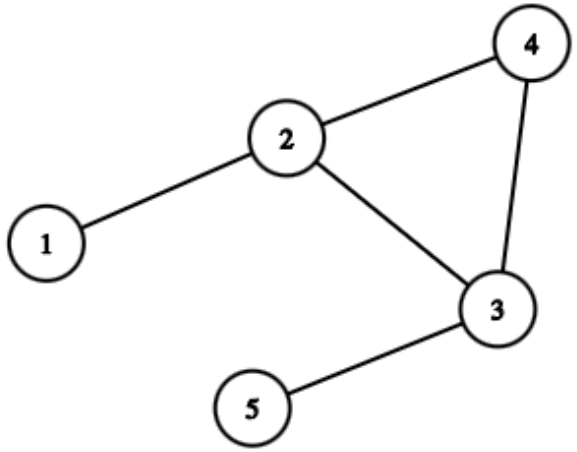
```cpp
// BJ 2164 cpp
#include <iostream>
#include <queue>

using namespace std;

int main() {
    cin.tie(0);
    ios::sync_with_stdio(false);

    //
    int N;
    queue<int> q;

    cin >> N;

    // queue에 1부터 N까지 push (1부터 빠져나가므로)
    for(int i = 1; i ≤ N; i++)
        q.push(i);

    // queue에 2개 이상의 데이터 남아있을때 반복
    while(q.size() > 1) {
        // 1. 한 장을 버린다 (pop)
        q.pop();

        // 2. 한 장을 뽑아서 맨 아래에 놓는다. (pop 후 push)
        int tmp = q.front();
        q.pop();
        q.push(tmp);
    }

    // queue에 남은 마지막 한 장 출력
    cout << q.front() << endl;
}
```

비선형 자료구조

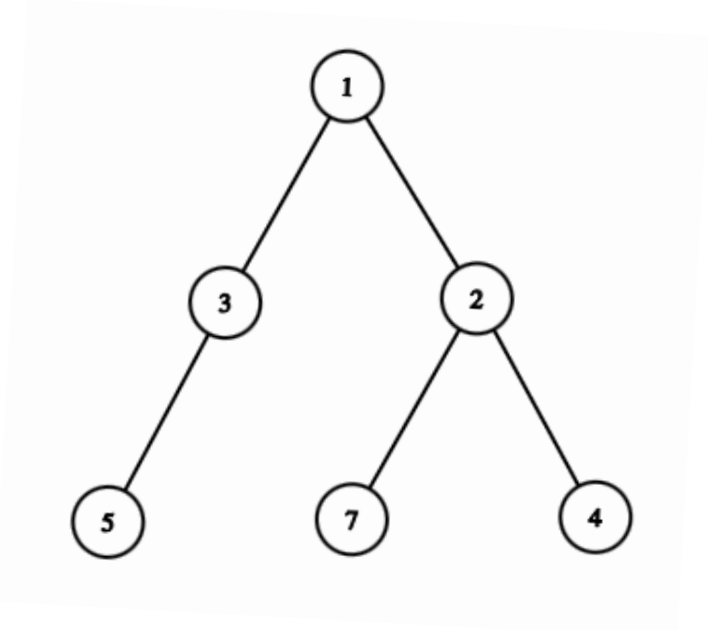
- 하나의 원소가 여러 개의 원소와 연결될 수 있는 자료구조
- 계층 (트리), 망 (그래프) 형태의 관계를 표현하는 자료구조

비선형 자료구조 - Graph

- Graph(그래프)란?
  - **정점**(Vertex)와 정점을 잇는 **간선**(Edge)로 구성된 자료구조
  - 도시-도시, 사람-사람 등 다양한 현상을 표현할 수 있는 자료구조
    - ※ 가령, 도시(정점)와 도시를 잇는 도로(간선)가 길 수록 가중치를 높게 부여해서 그래프로 표현 가능



- Tree(트리)란?
  - 다음 조건을 모두 만족하는 그래프를 트리라고 정의
  - 1) 연결 그래프이다
    - ※ 임의의 서로 다른 두 정점을 이어주는 경로가 존재한다.
  - 2) 사이클이 없다 (acyclic)
  - 디렉토리 구조, 이진 탐색 트리, 조직도 등 계층 구조 혹은 의사 결정 표현



비선형 자료구조

- C++ 에서의 그래프, 트리
  - 직접 해당 자료구조를 표현하는 STL 컨테이너는 없음
  - **인접 행렬**, 혹은 **인접 리스트**로 구현
    - ※ 자세한 구현과 이론은 6주차, 7주차, 8주차 강의자료에 수록할 예정

연관 배열

- 연관 배열(Associative Array)이란?
  - 하나의 키 값에 연관되는 값들을 1 : N 구조로 저장하거나, 키 값을 조회할 수 있는 자료구조
  - C++의 Associative Container에서 다양한 연관 배열 기능들을 제공
  - Associative Container는 자료를 key를 기준으로 **정렬된 상태를 유지**하며, BST(Balanced Binary Search Tree) 구조로 구현됨
    - ※ BST는 8장 트리 부분에서 자세히 설명할 예정

연관 배열 - set

- set이란?
  - set은 Associative Containers에서 제공하는 추상적 자료형
  - key를 **중복되지 않도록 저장**하는 자료형
  - 주로 값을 저장한 다음 **값의 존재 유무를 검색하기 위해 사용**
  - 삽입, 삭제, 검색 세가지 연산을 지원함 (세 연산 모두 O(log N))
    - ※ 세 연산의 시간 복잡도는 8주차 강의자료에 수록될 예정
    - ※ 원리가 궁금하면 BST에 대해 찾아보자.

연관 배열 – set

- set의 메소드
  - insert(key) : key를 set에 삽입
  - find(key) : key가 set에 존재하는지 검색. 만약 존재하지 않는다면 end() iterator 반환
  - erase(key) : set에서 key를 삭제. 인자로 iterator를 받을 수 있음
  - end() : set의 마지막 원소 다음 iterator를 반환
  - begin() : set의 시작 iterator를 반환

```cpp
#include <set>

int main() {
    set<int> a;

    // a에 순서대로 3, 1, 5, 5삽입
    a.insert(3);
    a.insert(1);
    a.insert(5);
    a.insert(5);

    // 중복값은 한번만 삽입되고, 키값이 내부에서 정렬되므로 1 3 5가 출력됨
    // 마지막 iterator 이후의 iterator가 a.end()이므로, 반복문 종료 조건으로 사용됨
    for(auto i = a.begin(); i != a.end(); ++i) {
        cout << *i << " ";
    }
    cout << endl;

    // erase에 iterator를 인자로 넘겨줌
    a.erase(a.begin());
    // erase에 키값을 인자로 넘겨줌
    a.erase(3);


    // a의 시작 iterator의 값 1과 3을 삭제했으므로 5만 출력됨
    for(auto i = a.begin(); i != a.end(); ++i) {
        cout << *i << " " ;
    }

    return 0;
}
```

```
1 3 5
5 %
```

수 찾기 BOJ 1920

- 문제
  - N개의 정수 A[1], ..., A[N]이 주어졌을 때, 정수 X가 이 안에 존재하는지 확인하라.

수 찾기 BOJ 1920

- 문제
  - N개의 정수 A[1], ..., A[N]이 주어졌을 때, 정수 X가 이 안에 존재하는지 확인하라.

- 풀이
  - 찾는 수가 총 M개 이므로, 배열로 구현해 순차 접근 시 O(N * M)의 시간 복잡도를 가진다.. 더 빠르게 처리할 순 없을까?
  - 문제는 정수 집합에서 X의 존재 유무를 M번 **검색**하는 것 !
  - 따라서 검색이 빠른 set을 사용 -> O(M log N)의 시간 복잡도로 처리할 수 있다.

```cpp
// BJ 1920 수 찾기
#include <iostream>
#include <vector>
#include <set>

using namespace std;

int main() {
    cin.tie(0);
    ios::sync_with_stdio(false);

    int N, M;
    // 집합 A 정의
    set<int> A;

    // N을 입력받고, 총 N개의 값 집합에 넣기
    cin >> N;
    for(int i = 0; i < N; i++) {
        int tmp;
        cin >> tmp;
        A.insert(tmp);
    }

    // M개의 값에 대하여 A에 검색하기
    cin >> M;
    while(M--) {
        int tmp;
        cin >> tmp;

        // 값이 있다면 1, 없으면 0 출력
        if(A.find(tmp) != A.end())
            cout << "1\n";
        else
            cout << "0\n";
    }
}
```

추상적 자료형 - map

- map이란?
  - map은 Associative Containers에서 제공하는 추상적 자료형
  - key와 key에 대응되는 데이터를 **(key, value)** 형태로 저장한 자료구조
  - set과 마찬가지로 **검색과 조회를 빠르게 하기 위해 사용**됨
  - (학번, 이름) 과 같이 키 값에 대응되는 값의 조회가 자주 일어나는 상황에서 사용하기 좋은 자료구조
  - set과 마찬가지로 insert, erase, find 세 가지 기본 연산을 지원 (O(log N))

추상적 자료형 – map

- map의 메소드
  - insert({key, value}) : key와 key에 대응되는 value를 map에 입력
  - find(key) : key에 대응되는 value 검색. 만약 key가 없다면 end() iterator 반환
  - erase(key) : key에 대응되는 (key, value) 쌍을 제거. key 대신 iterator를 인자로 받을 수 있음
  - end() : 마지막 원소 다음의 iterator 반환
  - begin() : 시작 원소의 iterator 반환

```cpp
#include <map>

int main() {

    // key, value의 데이터 타입을 제너릭으로 받아서 정의 가능
    map<string, int> mp;

    // mp에 pair로 (key, value)를 입력
    mp.insert(make_pair("하승민", 94));
    mp.insert(make_pair("이명준", 96));
    mp.insert(make_pair("이지민", 99));
    mp.insert(make_pair("이정주", 93));

    // [] 연산자로 key에 대응되는 value에 대입 연산
      mp["이명준"] = 85;

    // key값을 인자로 전달해 ("하승민", 94) 데이터 삭제
    mp.erase("하승민");

    // 내부는 key값 기준으로 오름차순 정렬되므로, 사전순으로 출력
    for(auto i = mp.begin(); i != mp.end(); ++i) {
        cout << i->first << " " << i -> second << "\n";
    }
    return 0;
}
```

```
이명준 85
이정주 93
이지민 99
```

비밀번호 찾기 BOJ 17219

- 문제
  - N개의 사이트 주소와 비밀번호가 주어진다. M개의 사이트 주소가 주어질 때, 각각 알맞는 비밀번호를 출력하라.

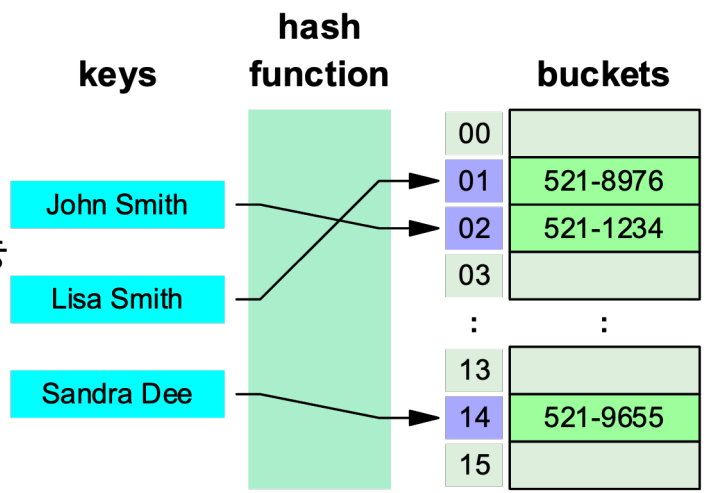
비밀번호 찾기 BOJ 17219

- 문제
  - N개의 사이트 주소와 비밀번호가 주어진다. M개의 사이트 주소가 주어질 때, 각각 알맞는 비밀번호를 출력하라.
- 풀이
  - N 개의 사이트 주소에 대응하는 비밀번호를 구현해야 함.
  - N과 M의 크기는 각각 100,000 이므로, O(N * M) (순차 검색) 시간 복잡도라면 시간 초과 발생 ..
  - 이때 map의 검색 시간 복잡도는 O(log N)이므로, O(M log N) 시간 복잡도로 풀이 가능 !
  - key를 사이트 주소, value를 비밀번호로 설정하여 map을 생성

```cpp
// 백준 17219 비밀번호 찾기
#include <iostream>
#include <map>
using namespace std;

int main() {
    cin.tie(0);
    ios::sync_with_stdio(false);

    int N, M;
    // 사이트 주소 (문자열) - 비밀번호 (문자열) 로 매핑되는 map 정의
    map<string, string> mp;

    cin >> N >> M;

    // 맵에 {주소, 비밀번호} 저장
    for(int i = 0; i < N; i++) {
        string addr, pw;
        cin >> addr >> pw;

        mp.insert(make_pair(addr, pw));
    }

    // 총 M번 입력받은 주소의 pw검색 후 출력
    for(int i = 0; i < M; i++) {
        string addr;
        cin >> addr;

        cout << mp[addr] << '\n';
    }
}
```

연관 배열 – hash table

- 해시 함수
  - key값을 연산을 통해 고정 길이의 해시 값 혹은 인덱스를 생성
  - 임의 길이의 키를 고정 길이 값으로 매핑하며, 출력 공간이 유한하여 충돌 발생 가능
    - ※ 상태공간이 사실상 무한한 입력에서 낮은 쪽으로 매핑하면 비둘기집 원리에 의해 반드시 충돌이 발생함
  - 입력 공간 U에 대해 어떤 출력 공간 R로 매핑하는 함수
    - ※ 주로 정수 인덱스 혹은 고정 길이의 해시 값으로 매핑



- 해시 테이블이란?
  - key를 해싱해 정수(인덱스)로 만들고, 해당 인덱스에 해당하는 버킷에 매핑하는 자료구조
    - ※ 버킷은 해시 테이블의 한 칸(인덱스)로, 충돌한 데이터들의 저장 단위
  - 체이닝, 오픈 어드레싱 등 충돌을 처리하는 규칙과 함께 정의 됨
    - ※ 체이닝은 linked list로 충돌 된 데이터들을 관리
  - 삽입, 삭제, 검색 세가지 연산을 **O(1) 수준으로 제공**

연관 배열 – hash table

- hash table의 조회
  - 정렬되지 않은 길이 N 배열에서 key 조회 -> **O(N)**.
  - 데이터가 N, 버킷이 M개인 hash table에서 조회할 경우
  - 1) key를 해싱하여 얻은 인덱스로 이동
  - 2) 버킷당 평균 원소 수 **a = N / M** -> 최대 a개의 데이터를 조회하면 됨
    - ※ 체이닝 방법으로 충돌 데이터들을 관리할 경우
  - 내부적으로 a가 상수 수준의 크기가 되도록 M을 조절하므로 **O(1 + a) = O(1)**

연관 배열 – hash table

- hash table 의 삽입과 삭제
  - 데이터가 N, 버킷이 M개인 hash table에서 삽입할 경우
  - 버킷이 여유로울 경우 : 해싱 비용 + 버킷 삽입 비용 = **(O(1))**
    - ※ 해싱 알고리즘 자체는 key의 모든 비트를 한번씩 읽어야 하므로 하한이 O(비트 길이 수)
    - ※ PS에서는 key가 주로 int형 (32비트) 이므로 상수로 취급해도 무관함
  - 버킷이 가득 찰 경우 : M의 크기를 조절해야함 **(O(N))**
  - 따라서 삽입의 시간 복잡도는 **amortized O(1)**
    - ※ Amortized는 알고리즘을 여러번 수행할 때의 평균 시간 복잡도의 상한선을을 의미
  - 마찬가지로 데이터의 삭제 또한 **amortized O(1)**

연관 배열 – hash table

- hash table 시간 복잡도
  - 조회 : O(1)
  - 삽입 : amortized O(1)
  - 삭제 : amortized O(1)
- 특징 및 단점
  - 역연산 불가능 : 해시 함수는 key를 고정 길이로 변환 + 복잡한 연산 -> 정보 손실 발생 및 역연산 어려움 발생
  - 충돌 발생 : 서로 다른 key가 동일한 해시 값으로 변환될 수 있음 -> 충돌이 많을 수록 성능 저하
  - 순서 or 범위 검색 불가능 : 해시 값은 삽입 삭제 조회가 굉장히 빠르지만, 정렬되어 있지 않음

연관 배열 – hash table

- C++ 에서의 hash table
  - std::unordered_map, std::unordered_set의 내부가 hash table로 구현됨
  - 문제가 값의 존재 여부, 빈도만 요구할 때 -> unordered_map/set
  - 문제가 데이터가 정렬 상태로 저장되는 것을 요구할 때-> map/set

연관 배열 – hash table

- C++ 에서의 hash table
  - 가령 앞선 문제 비밀번호 찾기 (BOJ 17219)은 {사이트 주소, 비밀번호} 가 내부적으로 정렬될 필요 x, 삽입과 조회가 문제 풀이를 위한 주된 연산
  - 따라서 map 대신 unordered_map을 사용하여 향상된 속도 O(1)로 삽입, 조회 가능

103557938	ccy0111	4 17219	맞았습니다!!	16056 KB	100 ms	C++17 / 수정	712 B	1분 전
103557842	ccy0111	4 17219	맞았습니다!!	16284 KB	188 ms	C++17 / 수정	500 B	7분 전

※ 위가 같은 알고리즘에 대하여 자료구조만 unordered_map을 사용한 경우.

  - **따라서 문제가 요구하는 연산의 성질을 파악하고, 적절한 자료구조를 선택하는 것이 중요!**

그 외 연습문제

- 문제가 어떠한 연산을 요구하는지 잘 관찰하고, 앞서 배운 자료구조와 ADT를 적절히 활용해보는 연습을 해보자
- 3052 나머지
- 11866 요세푸스 문제 0
- 9375 패션왕 신혜빈
- 2015 수들의 합